

JDBC Fundamentals - Quick Revision Notes

1. How Java Communicates with a Database

Java app and MySQL are two separate programs (Java runs in JVM, MySQL runs as its own server). Java needs a special mechanism (JDBC) to connect, send SQL, get response, and convert it into usable data.

2. JDBC API and JDBC Driver

JDBC = Java Database Connectivity, a standard API to talk to any database. JDBC API gives common interfaces (Connection, Statement, ResultSet etc.), and JDBC Driver is the database-specific translator (e.g., MySQL Connector/J for MySQL).

3. Installing MySQL Server and DBeaver

MySQL Server, DBeaver, and JDBC driver are 3 different things — Server stores data, DBeaver is just a GUI client to view/run SQL, and Connector/J lets Java code talk to MySQL. Installing DBeaver does NOT start MySQL server.

4. Creating the JDBC Project

Simple Maven project structure (pom.xml + src/main/java). Add MySQL Connector/J dependency in pom.xml to enable JDBC connectivity — no Spring Boot needed for raw JDBC.

5. Creating the Database and Table

Use `CREATE DATABASE` and `CREATE TABLE` SQL commands to set up `student_db` and `students` table. Also create a plain Java `Student` model class (POJO) with getters/setters to hold row data.

6. Establishing a Database Connection

`Connection` interface represents an active session with DB, created using `DriverManager.getConnection(url, username, password)`. Modern JDBC (4.0+) auto-loads the driver, so `Class.forName()` is no longer required.

7. Executing SQL with JDBC

A `Statement` object (from Connection) is used to run SQL. Three methods: `executeQuery()` for SELECT (returns ResultSet), `executeUpdate()` for INSERT/UPDATE/DELETE (returns rows affected), `execute()` for unknown/mixed result types (returns boolean).

8. Understanding ResultSet

`ResultSet` is NOT a list — it's a cursor over rows returned by SQL. Cursor starts before the first row; call `.next()` to move forward and read column values manually using `getString()`, `getInt()`, etc. Prefer explicit column names over `SELECT *`.

9. Why PreparedStatement Is Preferred

Building SQL by string concatenation is unsafe (SQL injection, quote issues).

`PreparedStatement` uses `(?)` placeholders for values, keeping SQL structure separate from data — safer and cleaner. Note: placeholders protect values only, not table/column names.

10. Complete CRUD Operations

- **Create:** `INSERT` via `PreparedStatement`, use `RETURN_GENERATED_KEYS` to get auto-generated ID.
- **Read One:** `SELECT ... WHERE id=?`, wrap result in `Optional<Student>`.
- **Read All:** Loop through `ResultSet` with `while(resultSet.next())`, add each row to a List.
- **Update:** `UPDATE ... WHERE id=?`, check rows affected — don't call `findById()` before update (extra round trip + race condition risk).
- **Delete:** `DELETE FROM ... WHERE id=?`, check rows affected to know if it worked.

11. Closing JDBC Resources

Resources have parent-child order: `Connection` → `Statement` → `ResultSet`. Close in reverse order. Use **try-with-resources** (since these classes implement `AutoCloseable`) instead of manual `finally` blocks — much cleaner and safer.

12. Common Setup Problems

- **Connection refused** → check MySQL server running, host/port correct.
- **Access denied** → check username/password/permissions.
- **Unknown database** → create the database first.
- **DBBeaver "Public Key Retrieval" error** → enable `allowPublicKeyRetrieval=true` in driver properties.
- **Duplicate email** → happens due to `UNIQUE` constraint violation, expected behavior.

13. Key Takeaways (Cheat Sheet)

- Java app & DB server are separate — JDBC bridges them.
 - `Connection` = session, `Statement` = plain SQL, `PreparedStatement` = parameterized SQL.
 - `executeQuery()` → `SELECT`, `executeUpdate()` → `INSERT/UPDATE/DELETE`.
 - `ResultSet` cursor starts before row 1; must map rows to objects manually.
 - Always prefer `PreparedStatement` + `try-with-resources` for safety and clean code.
 - Raw JDBC is repetitive on purpose — this is why tools like Spring JDBC, JPA, Hibernate exist later.
-

 One-Line Full Flow (Memorize This)

Java App → JDBC API → JDBC Driver (Connector/J) → MySQL Server → SQL runs → Result/Rows sent back → Driver converts → ResultSet → Java manually maps to objects.